

HW 1: Free-fall fit

Physics Programming, Week 1 — Core

Core homework: infer gravity from free-fall measurements

We investigate whether measured fall distances agree with release from rest under approximately uniform gravity. We take distance positive downward, so $y(t) = \frac{1}{2}gt^2 + b$, with $g > 0$ in m/s^2 and offset b in m . The supplied transformation `t_s .^ 2` makes squared time the horizontal coordinate. We fit $y = at^2 + b$ and interpret $g = 2a$; the factor of two comes from the motion equation, not from a software convention.

We work in these stages:

1. Inspect the supplied `free_fall.csv`: identify the time and distance columns, their units, and the number of observations.
2. Predict the shape of y versus t and of y versus t^2 before running the fit. Apply the direct least-squares fit below and retain the predictions and measured-minus-predicted residuals.
3. Draw observations as distinct markers, overlay the fitted model, and make a separate residual plot with a zero reference. Label every axis with its quantity and units.
4. Save the required PNG/PDF artifacts and report the fitted g and offset. Explain whether the residuals support the model over the measured interval and name one assumption that the fit cannot establish.
5. Run the published exact-line check and submit the numerical fragment and figure/report artifacts specified on the assignment sheet.

We submit a fragment that uses the supplied input and output names, and the grader supplies the function wrapper around it; defining a Julia function is not part of Week 1. Keep this assignment sheet nearby, because it is where the exact public tests, tolerances, domains, and filenames are published. Rounding a displayed answer must not round the values used for predictions or grading.

Student physics box: downward free fall

We measure downward distance y from the release point, so downward is positive. For a dropped object over a short distance with negligible air drag and approximately constant gravitational acceleration, $y = at^2 + b$, where t is seconds, y metres, a is m/s^2 , and $g = 2a$ is the positive gravity magnitude. At $t = 0$, $y = b$; an ideal origin has $b = 0$. A timing/distance offset, drag, or changing acceleration can produce residual structure or a nonzero intercept.

Program and artifact workflow

Write a short Julia program that reads the supplied `reveal/week01/free_fall.csv`. A CSV is a plain text table whose values are separated by commas; the first row gives column names. The program reads its `t_s` and `y_m` columns, fits the line directly, and saves an observations/model figure and a residual figure as PNG and PDF. We submit this program (or its numerical fragment) together with the figures. Independent CSV tables and plotting return in Week 7.

What we submit to the graded fragment

This Week 1 question is a fragment. The platform supplies valid `t_s` and `y_m` vectors. Submit statements only: no function, return, validation, printing, file reading, or plotting. Leave `g`, `intercept`, `predicted`, and `residual`. The required new expression `t_s .^ 2` squares every time separately; the dot makes the supplied operation element by element. Each identifier stays unit-free; the unit belongs in a trailing comment.

```
t2 = t_s .^ 2 # s^2
t2_bar = sum(t2) / length(t2) # s^2
y_bar = sum(y_m) / length(y_m) # m
a = sum((t2 .- t2_bar) .* (y_m .- y_bar)) / sum((t2 .- t2_bar) .^ 2) #
    m/s^2
intercept = y_bar - a * t2_bar # m
g = 2 * a # m/s^2
predicted = a .* t2 .+ intercept # m
residual = y_m .- predicted # m
```

Required evidence

Run your program on the supplied CSV. Report *g*, *b*, the units, whether residuals look randomly scattered or patterned, and the saved PNG/PDF paths. Do not claim precision beyond the measurements.

How is this exercise checked?

Here is what the platform does when we submit. It places our fragment inside a supplied function—a wrapper named `freefall_fit(t_s, y_m)`—that calls our code and returns the four required values. Before our fragment runs, the wrapper’s public validation checks the vectors it passes in: vector shape, finite non-Boolean real values, nonnegative times, and nonzero squared-time spread. In Week 1 our fragment is only ever called with valid inputs, so we can put our attention on the fitting itself. The numeric checks then compare the results with `isapprox` using `atol=rtol=1e-10`, and because the grader receives the data directly, nothing here assumes a local file or a plot. The staff tests use these same published rules, so the public checks give us a faithful preview of the grading.

```
g, intercept, predicted, residual =
    freefall_fit([0.0, 1.0, 2.0], [1.0, 5.0, 17.0])
@test isapprox(g, 8.0; atol=1e-10, rtol=1e-10)
@test isapprox(intercept, 1.0; atol=1e-10, rtol=1e-10)
@test all(isapprox.(predicted, [1.0, 5.0, 17.0]; atol=1e-10, rtol=1e-10)
)
@test all(isapprox.(residual, [0.0, 0.0, 0.0]; atol=1e-10, rtol=1e-10))
```

Optional advanced note

The wrapper and its `@test` checks are the platform’s own machinery, and none of it is required Week 1 syntax. A wrapper is simply a supplied function that calls our code and returns its results; a test written with `@test` states a claim that has to hold, and its companion `@test_throws` states that an invalid input must be rejected. Reading them shows us why the same fragment gets the same verdict each time. We submit only the fragment above.